

Database Development with PL/SQL INSY 8311



ADVENTIST UNIVERSITY
OF CENTRAL AFRICA

Instructor:

- Eric Maniraguha | ericmaniraguha2024@gmail.com | [LinkedIn Profile](#)

6h00 pm – 8h50 pm

- Monday A -G209
- Tuesday B-G209
- Wednesday E-G209
- Thursday F-G308

September 2024

GitHub: Where your code meets its social network!





Reference reading

- [YouTube Tutorial: **Git Tutorial for Beginners: Learn Git in 1 Hour**](#)
- [GitHub and Git Tutorial for Beginners](#)
- [Introduction to GitHub](#)
- [Intro to GitHub for version control](#)
- [DevOps Tools Mapping](#)
- [How to Design an Attractive GitHub Profile Readme...](#)
- [YouTube Tutorial: **How to create Professional Github Readme Profile \(Step By Step\)**](#)
- [Getting Started with R Markdown](#)
- [YouTube Tutorial: **Git Merge Conflicts | How Merge Conflicts Happen | How to resolve Merge Conflicts | Merge Conflict**](#)

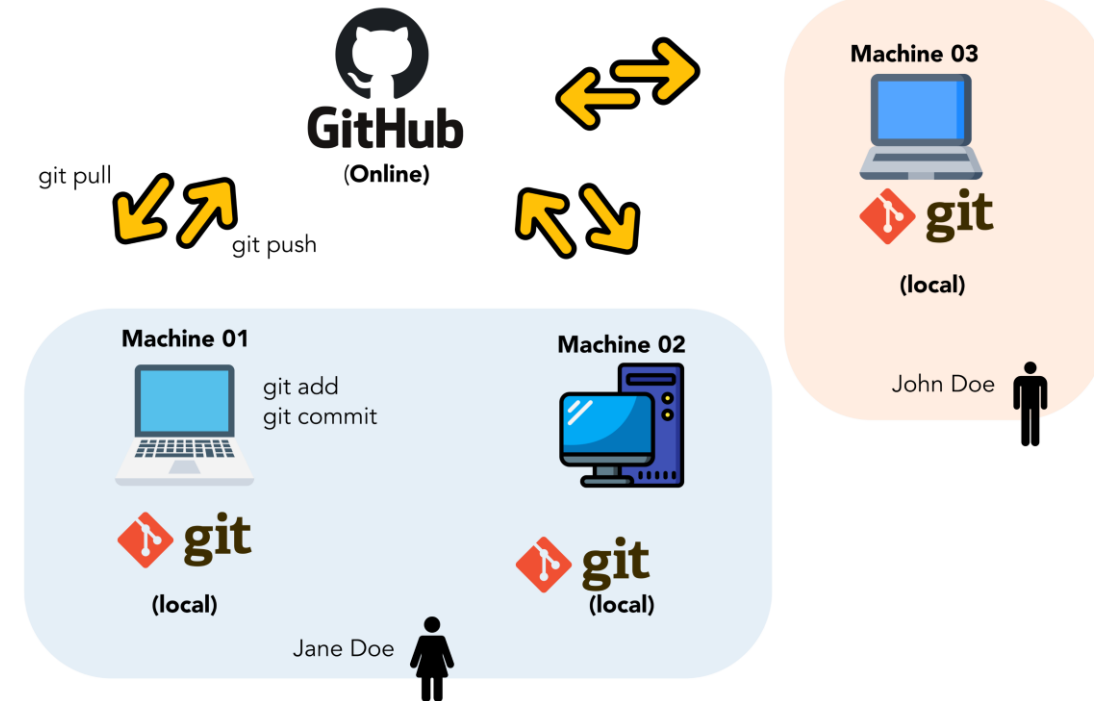
Lecture 02 – Introduction to GitHub

Introduction



Course Overview

This course is designed to introduce students to GitHub, a powerful platform for version control and collaboration. Students will learn the basics of using GitHub, including creating repositories, committing changes, branching, merging, and collaborating with others. By the end of the course, students will be comfortable using GitHub for their projects.





Course Objectives

- **Understand Version Control:** Learn the importance of version control for tracking changes, preventing data loss, and enabling collaboration.
- **Create and Manage Repositories:** Gain skills in setting up and organizing GitHub repositories for your projects.
- **Make and Track Commits:** Master the process of committing changes to your projects, ensuring a clear history of your work.
- **Use Branching and Merging:** Learn to manage different versions of your project with branching and merging, essential for handling features, bug fixes, and collaboration.
- **Collaborate on GitHub:** Utilize GitHub's collaboration tools, including pull requests, forks, and issues, to work effectively in a team.

Key Features and Benefits of Using GitHub



Feature	Description
Version Control	GitHub uses Git, a powerful version control system, allowing you to track changes in your projects over time. This means you can revert to previous versions, see what changes were made, and by whom.
Collaboration	GitHub makes it easy for multiple people to work on the same project simultaneously. Features like pull requests and issues allow for seamless collaboration, code review, and project management.
Community and Open Source	GitHub hosts millions of projects, making it the largest repository of open-source code. This provides access to a vast library of resources, allowing you to contribute to projects, learn from others, and showcase your work.
Integration and Automation	GitHub integrates with numerous tools and services, enhancing your workflow. GitHub Actions, for instance, allows you to automate tasks like testing, deployment, and continuous integration.
Professional Portfolio	For students and professionals, GitHub serves as a portfolio to display your work. Many employers use GitHub profiles to evaluate candidates, making it an essential tool for career development.
Documentation and Project Management	GitHub includes features like wikis and project boards, which help you document your projects and manage tasks, making it a comprehensive tool for both individual and team projects.

What is version control?



Version control

- Manages and tracks changes in your work (e.g., data, code, notes).

Traditional Method

- Saving multiple versions as separate files (e.g., Dissertation_script_25thFeb.R) can be inefficient and difficult to compare.

Git

- Streamlines version control, making it easier to handle different versions.

GitHub

- Offers online storage and backup, supporting collaboration and access.

Advanced Features

- Git includes command-line tools for complex tasks; further learning resources are available.

Getting Started

- Quickly set up version control with RStudio and GitHub for immediate use.

Unlocking Git: Understanding Commands Through Diagrams

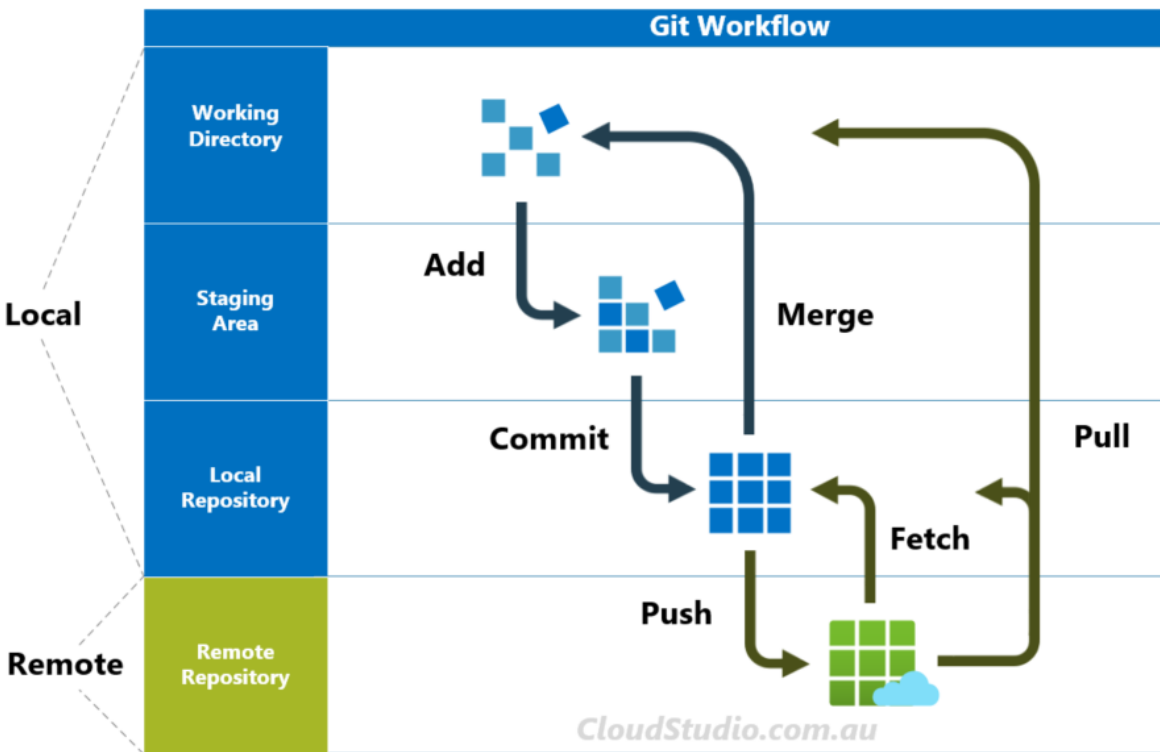


Image Source: <https://cloudstudio.com.au/2021/06/26/git-command/>

1

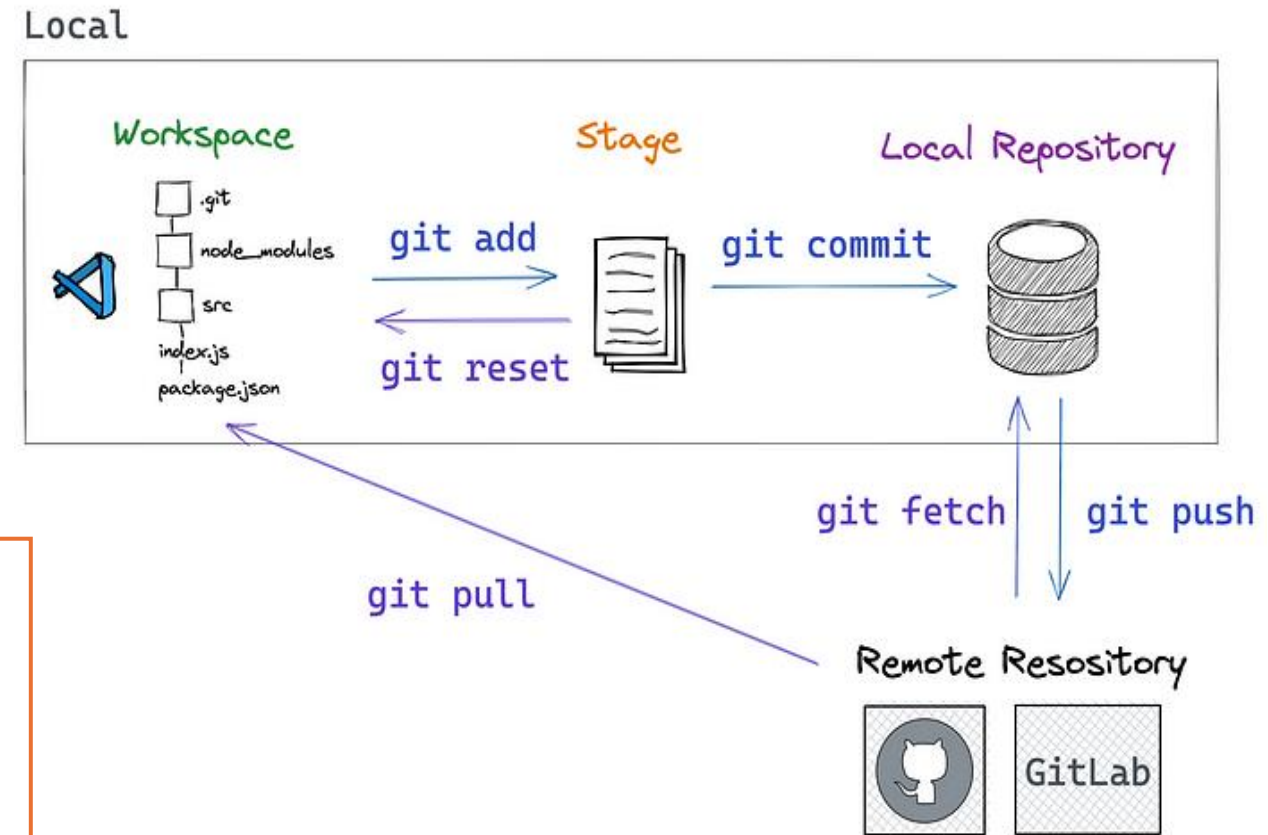


Image Source: <https://programming.earthonline.us/you-can-master-git-git-commands-with-these-diagrams-40a0b2f5cc42>

2

How to get started



- Please register on the [GitHub website](#).
- On your computer, you need to install [Git](#) first. The process will depend on your operating system: please follow the instructions below by clicking the relevant button.



.....
.....
.....
.....

.....
.....
.....
.....



Basic Commands

The basic commands include initializing the Git repository, saving changes, checking logs, pushing the changes to the remote server, and merging.



Command	Description
<i>git init</i>	Create a Git repository in a local directory.
<i>git clone <remote-repo-address></i>	Copy the entire repository from a remote server to a local directory. Can also clone local repositories.
<i>git add <file.txt></i>	Add a single file or multiple files and folders to the staging area.
<i>git commit -m "Message"</i>	Create a snapshot of changes and save it in the repository.
<i>git config</i>	Set user-specific configurations like email, username, and file format.
<i>git status</i>	Show the list of changed files or files that have yet to be staged and committed.
<i>git push <remote-name> <branch-name></i>	Send local commits to the remote branch of the repository.
<i>git checkout -b <branch-name></i>	Create a new branch and switch to it.
<i>git remote -v</i>	View all remote repositories.
<i>git remote add <remote-name> <host-or-remoteURL></i>	Add a remote server to the local repository.
<i>git branch -d <branch-name></i>	Delete a branch.
<i>git pull</i>	Merge commits from a remote repository into a local directory.
<i>git merge <branch-name></i>	After resolving merge conflicts, blend the selected branch into the current branch.
<i>git log</i>	Show a detailed list of commits for the current branch.

GitHub User Profile



- **Profile README:** Displayed at the top of your GitHub profile.
- **Full Control:** Customize your README to present yourself as you wish.

What to Include:

- **About Me:** Describe your work and interests.
- **Contributions:** Highlight projects you're proud of with relevant context.
- **Community Guidance:** Offer tips on getting help in communities you're active in.
- **Text Formatting:** Use GitHub Flavored Markdown to format text and include emojis, images, and GIFs in your profile README.
- **Learn More:** Refer to "Getting started with writing and formatting on GitHub."
- **Customization Guide:** Check out "Quickstart for writing on GitHub" for a hands-on guide to customizing your profile README.

The screenshot shows a GitHub profile README for Eric Maniraguha. It includes a profile picture, a bio, a list of skills, and various statistics.

Hi 🌟, I'm Eric Maniraguha

A passionate Data Science and Machine Learning enthusiast from Kigali, Rwanda

Profile views: 2,300

FOLLOW @MANIRAGUHA_ERIC

Ask me about ArcGIS, PowerBI, Google Earth Engine, Deep Learning, LaTeX, GCP, AWS, GitHub, Kaggle, Colab, ETL, MLOps

Connect with me:

Skills:

1. Data Science and Machine Learning Tools:

2. Languages and Tools:

Skills:

Most Used Languages

Language	Percentage
Python	82.75%
JavaScript	1.10%
CSS	3.04%
Java	2.04%
HTML	1.10%
JavaScript	0.72%
Python	0.37%

Eric Maniraguha's GitHub Stats

Stat	Value
Total Stars Earned	6
Total Commits (2024)	250
Total PRs	186
Total Issues	0
Contributed to (last year)	6

1,226 Total Contributions
Oct 22, 2018 - Present

0 Current Streak
Aug 24

9 Longest Streak
Feb 4, 2022 - Feb 12, 2022

About your profile README

GitHub User Profile



  New repository

Create a new repository

A repository contains all project files, including the revision history. Already have a project repository elsewhere? [Import a repository.](#)

Required fields are marked with an asterisk ().*

Repository template

No template ▾

Start your repository with a template repository's contents.

Owner *

 ericmaniraguha ▾

Repository name *

Great repository names are short and memorable. Need inspiration? How about [didactic-meme](#) ?

Description (optional)

☒  **Public**

Anyone on the internet can see this repository. You choose who can commit.

☐  **Private**

You choose who can see and commit to this repository.

Adding a profile README

In the upper-right corner of any page, select + , then click **New repository**.

Initialize this repository with:

☐ **Add a README file**

This is where you can write a long description for your project. [Learn more about READMEs.](#)

Add .gitignore

.gitignore template: None ▾

Choose which files not to track from a list of templates. [Learn more about ignoring files.](#)

Choose a license

License: None ▾

A license tells others what they can and can't do with your code. [Learn more about licenses.](#)

 You are creating a public repository in your personal account.

Create repository

Removing a profile README



The profile README will be removed from your GitHub profile if any of the following apply:

- The README file is removed or made empty.
- The repository is made private.
- The repository name no longer matches your username due to a change in either or both names.

The method you choose depends upon your needs, but if you're unsure, we recommend making your repository private. For steps on how to make your repository private, see "[Setting repository visibility](#)."

What is a repository?

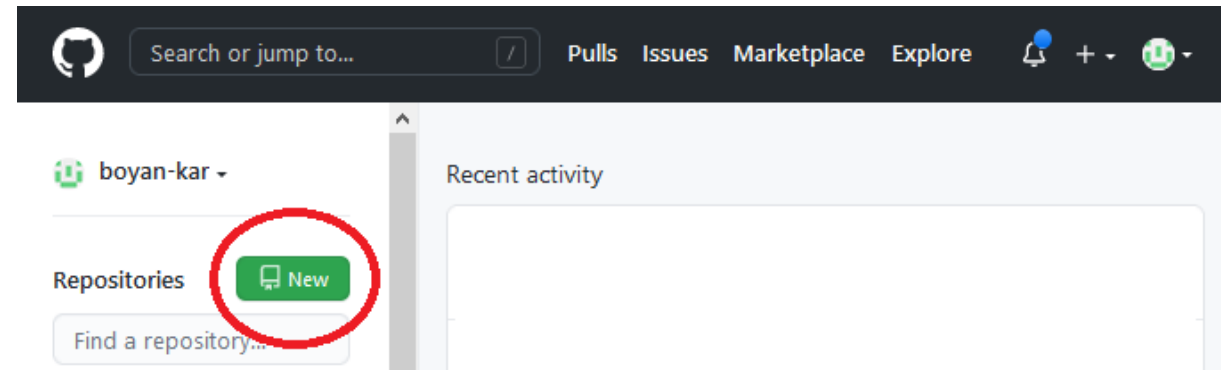


1. Create your own repository and project folder structure

Think of a repository (or repo) as the "main folder" for a project, where everything related to that project is stored. A repo can contain multiple folders or individual files, all organized under one project.

You'll maintain both a local copy (on your computer) and an online copy (on GitHub) of all the files in your repository.

The typical GitHub workflow is captured by the phrase "**commit-pull-push**."



What is a repository?



Create a new repository

Repository template

Start your repository with a template repository's contents.

No template ▾

Owner *



boyan-kar ▾



Repository name *

my-first-repository ✓

Great repository names are short and memorable. Need inspiration? How about [symmetrical-couscous?](#)

Description (optional)

☐  **Public**
Anyone on the internet can see this repository. You choose who can commit.

☒  **Private**
You choose who can see and commit to this repository. 

Here's a concise version for a PowerPoint slide:

Initialize Repo:

- Create a *README.md* for project details, purpose, and licensing.

Markdown Format:

- Use *.md* for structured project documentation.

.gitignore File:

- Exclude unnecessary files from the repo.

Language Template:

- Select the relevant template (e.g., R) in the *.gitignore*.

Repository is created



 **my-first-repository** Public Pin Unwatch 1 Fork 0 Star 0

 **Start coding with Codespaces**
Add a README file and start coding in a secure, configurable, and dedicated development environment.
[Create a codespace](#)

 **Add collaborators to this repository**
Search for people using their GitHub username or email address.
[Invite collaborators](#)

Quick setup — if you've done this kind of thing before

[Set up in Desktop](#) or [HTTPS](#) [SSH](#) <https://github.com/ericmaniraguha/my-first-repository.git> 

Get started by [creating a new file](#) or [uploading an existing file](#). We recommend every repository include a [README](#), [LICENSE](#), and [.gitignore](#).

...or create a new repository on the command line

```
echo "# my-first-repository" >> README.md
git init
git add README.md
git commit -m "first commit"
git branch -M main
git remote add origin https://github.com/ericmaniraguha/my-first-repository.git
git push -u origin main
```



...or push an existing repository from the command line

```
git remote add origin https://github.com/ericmaniraguha/my-first-repository.git
git branch -M main
git push -u origin main
```



After creating **my-first-repository**, I can push my code from local repository to remote repository

Repository is created



Explanation:

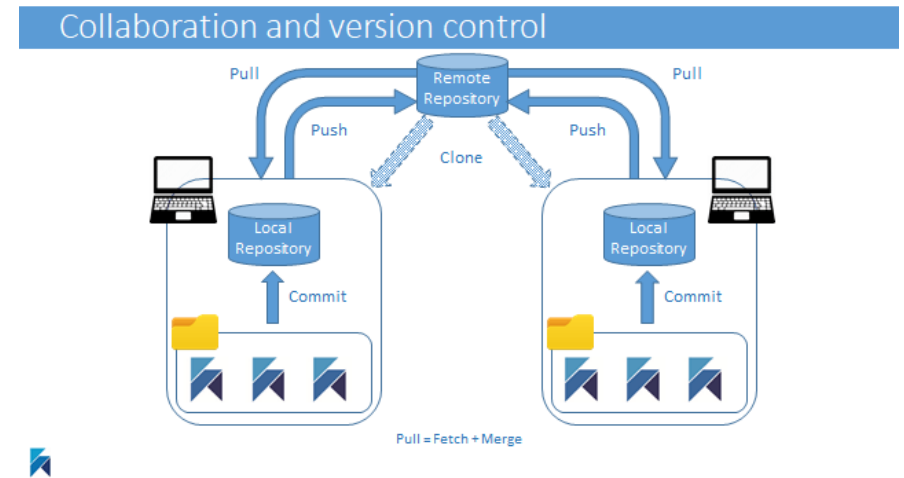
- `echo "# my-first-repository" >> README.md`: This creates a new README.md file with a simple title inside it.
- `git init`: Initializes a new Git repository in your current directory.
- `git add README.md`: Stages the README.md file for committing.
- `git commit -m "first commit"`: Commits the staged file with the message "first commit."
- `git branch -M main`: Renames the current branch to main.
- `git remote add origin <url>`: Links the local repository to a remote GitHub repository.
- `git push -u origin main`: Pushes the code from the local main branch to the remote main branch and sets up tracking between the two.

Use case: This method is useful when you're starting from scratch and want to create a new Git repository, initialize it, and then push it to GitHub.

Explanation:

- `git remote add origin <url>`: Links the local repository to the remote GitHub repository.
- `git branch -M main`: Renames the branch to main (if it hasn't been done already).
- `git push -u origin main`: Pushes the local branch to the remote repository and sets up the tracking.

Use case: This method assumes that you've already initialized a Git repository, committed your changes, and just want to push an existing repository to GitHub.



Why “.gitignore”?



The **.gitignore** file is essential because it tells Git which files or directories to ignore in a project.

This helps to:

- **Keep Repositories Clean:** Exclude unnecessary files (e.g., temporary files, build artifacts) from being tracked by Git.
- **Protect Sensitive Information:** Prevent sensitive data (e.g., API keys, passwords) from being accidentally committed.
- **Optimize Collaboration:** Ensure that only relevant files are shared among team members, reducing clutter and potential conflicts.
- **Maintain Efficient Version Control:** Focus on tracking and managing only the files that are important to the project.

```
1 # Demo/example
2
3 *.zip
4 *.gz
5 log/*.log
6 log/*.log.[0-9]
7 assests/videos/
8 !assests/video/iphone_*.mp4
9
```

How to create a git repository?



☐ **Public**
Anyone on the internet can see this repository. You choose who can commit.

☒ **Private**
You choose who can see and commit to this repository.

Initialize this repository with:

Skip this step if you're importing an existing repository.

☒ **Add a README file**
This is where you can write a long description for your project. [Learn more.](#)

☒ **Add .gitignore**
Choose which files not to track from a list of templates. [Learn more.](#)

.gitignore template: R

☐ **Choose a license**
A license tells others what they can and can't do with your code. [Learn more.](#)

This will set `main` as the default branch. Change the default name in your [settings](#).

Create repository

Here is how the repository should look:

boyan-kar / my-first-repository Private

Code Issues Pull requests Actions Projects Security Insights Settings

main 1 branch 0 tags

Go to file Add file Code

boyan-kar Initial commit 691a5b4 now 1 commit

.gitignore	Initial commit	now
README.md	Initial commit	now

README.md GitHub automatically shows the README.md file of the current directory.

my-first-repository

View your commit history here.

GIT Push and Pull Tutorial



Note: Create a new repository like the previous slide as the 1st step

2. Open your Git Bash

- Git Bash can be downloaded in [here](#), and it is a shell used to interface with the operating system which follows the UNIX command.

3. Create your local project in your desktop directed towards a current working directory

```
MINGW64:/c:/Users/Dell/Downloads/FaceDetect-master

Dell@DESKTOP-03TH7J0 MINGW64 ~
$ pwd
/c/Users/Dell

Dell@DESKTOP-03TH7J0 MINGW64 ~
$ cd C:/Users/Dell/Downloads/FaceDetect-master

Dell@DESKTOP-03TH7J0 MINGW64 ~/Downloads/FaceDetect-master
$
```

4. Initialize the git repository

```
MINGW64:/c:/Users/Dell/Downloads/FaceDetect-master/FaceDetect-master

Dell@DESKTOP-03TH7J0 MINGW64 ~/Downloads/FaceDetect-master/FaceDetect-master
$ git init
Initialized empty Git repository in C:/Users/Dell/Downloads/FaceDetect-master/FaceDetect-master/.git/

Dell@DESKTOP-03TH7J0 MINGW64 ~/Downloads/FaceDetect-master/FaceDetect-master (master)
$ |
```

GIT Push and Pull Tutorial



5. Add the file to the new local repository

```
MINGW64:/c/Users/Dell/Downloads/FaceDetect-master/FaceDetect-master
Dell@DESKTOP-03TH7J0 MINGW64 ~/Downloads/FaceDetect-master/FaceDetect-master (master)
$ git status
On branch master

No commits yet

Changes to be committed:
  (use "git rm --cached <file>..." to unstage)

        new file:   FaceFinder.py
        new file:   README.md
        new file:   demo.jpg
        new file:   demo.py
        new file:   demo_result.png
        new file:   face_ds.py
        new file:   face_model
        new file:   tfac.py

Dell@DESKTOP-03TH7J0 MINGW64 ~/Downloads/FaceDetect-master/FaceDetect-master (master)
$
```

6. Commit the files staged in your local repository by writing a commit message

```
MINGW64:/c/Users/Dell/Downloads/FaceDetect-master/FaceDetect-master
Dell@DESKTOP-03TH7J0 MINGW64 ~/Downloads/FaceDetect-master/FaceDetect-master (master)
$ git commit -m "First Commit"
[master (root-commit) 1fc80a3] First Commit
 8 files changed, 365 insertions(+)
 create mode 100644 FaceFinder.py
 create mode 100644 README.md
 create mode 100644 demo.jpg
 create mode 100644 demo.py
 create mode 100644 demo_result.png
 create mode 100644 face_ds.py
 create mode 100644 face_model
 create mode 100644 tfac.py

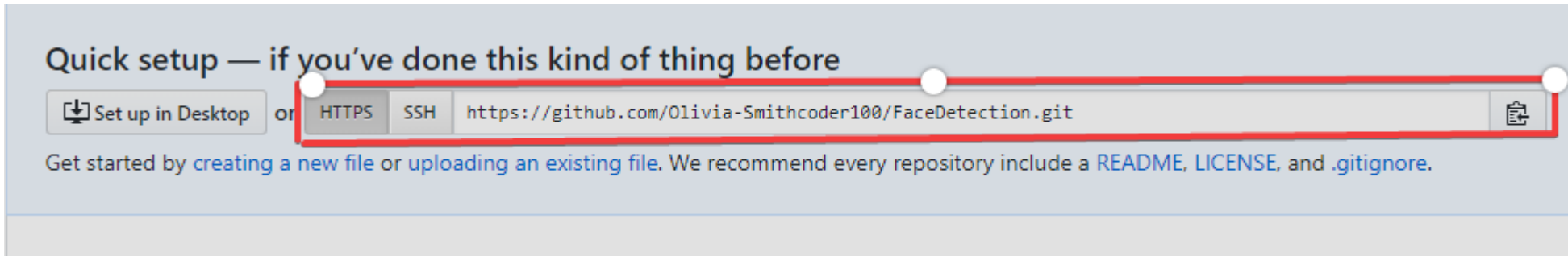
Dell@DESKTOP-03TH7J0 MINGW64 ~/Downloads/FaceDetect-master/FaceDetect-master (master)
$ |
```

GIT Push and Pull Tutorial



7. Copy your remote repository's URL from GitHub

- The HTTPS or URL is copied from the given GitHub account, which is the place of the remote repository.



8. Add the URL copied, which is your remote repository to where your local content from your repository is pushed

`git remote add origin 'your_url_name'`

GIT Push and Pull Tutorial



9. Push the code in your local repository to GitHub

- `git push -u origin master` is used for pushing local content to GitHub.
- In the code, the origin is your default remote repository name and '-u' flag is upstream, which is equivalent to '***set-upstream***.' and the master is the branch, name.upstream is the repository that we have cloned the project.
- Fill in your GitHub username and password.

A screenshot of a Windows terminal window with a black background and green text. The window title is 'MINGW64:/c:/Users/Dell/Downloads/FaceDetect-master/FaceDetect-master'. The terminal shows the following commands and output:

```
Dell@DESKTOP-03TH7J0 MINGW64 ~/Downloads/FaceDetect-master/FaceDetect-master (master)
$ git remote add origin https://github.com/Olivia-Smithcoder100/FaceDetection.git
Dell@DESKTOP-03TH7J0 MINGW64 ~/Downloads/FaceDetect-master/FaceDetect-master (master)
$ git push -u origin master
Username for 'https://github.com': Olivia-Smithcoder100
```

An 'OpenSSH' dialog box is overlaid on the terminal. It has a title bar with a feather icon and standard window controls. The text inside says 'Password for 'https://Olivia-Smithcoder100@github.com':'. Below the text is a white input field. At the bottom are 'OK' and 'Cancel' buttons.

Git Push and Pull Tutorial



10. View your files in your repository hosted on GitHub

- *You can finally see the file hosted on GitHub.*

A screenshot of a GitHub repository page for 'Olivia-Smithcoder100 / FaceDetection'. The page shows the repository name, a 'Watch' button with 0 notifications, a 'Star' button with 0 stars, and a 'Fork' button with 0 forks. Below this is a navigation bar with 'Code', 'Issues 0', 'Pull requests 0', 'Projects 0', 'Wiki', 'Security', 'Insights', and 'Settings'. A message states 'No description, website, or topics provided.' with an 'Edit' button. Below this is a summary bar showing '1 commit', '1 branch', '0 releases', and '1 contributor'. A bar at the bottom shows 'Branch: master' and a 'New pull request' button. To the right are buttons for 'Create new file', 'Upload files', 'Find File', and a green 'Clone or download' button. The main content area shows a list of files: 'FaceFinder.py', 'README.md', 'demo.jpg', 'demo.py', 'demo_result.png', 'face_ds.py', 'face_model', and 'tfac.py'. Each file is listed with its commit history ('First Commit') and the time since the commit ('5 minutes ago'). At the bottom is a 'README.md' file with an edit icon.

Challenge Yourself!

Elevate Your GitHub Expertise: Master Branching, Collaboration, and Conflict Resolution

Create Branches:

Learn how to create and manage branches to work on different features or fixes simultaneously.

Manage Team Projects:

Understand how to set up and manage team projects for collaborative development.

Form Groups:

Learn how to organize and collaborate within groups on GitHub.

Resolve Conflicts:

Master conflict resolution techniques to handle and merge conflicting changes smoothly.

Enhance Code

Integration: Improve your skills in integrating code from various sources to maintain a cohesive and high-quality project.

Career Opportunities with GitHub Skills: Unlocking Roles in Development, Data, and More

- **Software Developer:** GitHub is crucial for version control and collaboration in software development. Developers use GitHub to manage code, track changes, and collaborate with teams.
- **DevOps Engineer:** DevOps roles involve managing development, testing, and deployment pipelines. GitHub plays a central role in source control, continuous integration (CI), and deployment automation.
- **Open Source Contributor:** Many companies and projects operate in open source, where GitHub is widely used for collaboration. Contributing to open-source projects can be a great way to build your portfolio and network.
- **Data Scientist/Analyst:** GitHub is used to store code, data, and models in data science projects. It's also valuable for collaborating with other data professionals on analysis projects.
- **Full-Stack Developer:** Both frontend and backend developers use GitHub to manage their codebases, ensuring the project is well-organized and contributions from different team members are easily tracked.
- **Machine Learning Engineer:** Version control is critical when developing machine learning models, and GitHub helps manage model code, experiments, and collaborations.
- **Project Manager (Tech):** GitHub can be used for project management, allowing you to track tasks, bugs, and feature requests using GitHub issues and project boards.
- **Technical Writer:** Writing documentation for open-source or internal projects hosted on GitHub is a valuable skill. Documentation helps other developers understand how to use and contribute to projects.
- **Cloud Engineer:** GitHub integrates with cloud platforms like AWS, Azure, and Google Cloud for managing infrastructure as code and deploying applications.
- **Quality Assurance (QA) Engineer:** QA engineers use GitHub to manage testing scripts, track bugs, and collaborate on test automation.





Conclusion 

Thank you!